



SH320 Data Engineering

Spring 2026 Semester

NumPy

③ Efficient Computation in NumPy

Hyun Ahn

School of Computing and Artificial Intelligence

hyunahn@hs.ac.kr



Core Libraries for Data Science



- **[W2] N-Dimensional Arrays**
 - NumPy arrays
 - Array manipulation
- **[W3] Efficient Computation in NumPy**
 - Universal functions
 - Advanced features

Data Engineering



3.1 유니버설 함수

Universal Functions (UFuncs)



- **유니버설 함수**: NumPy에서 제공하는 **벡터화 연산 함수**(`numpy.ufunc`)
 - 다차원 배열에 대한 **원소 별** ^{element-wise} 연산을 수행
 - **벡터화** ^{vectorization}: 루프 연산을 인터프리터 계층이 아닌 **컴파일 계층에서 수행**
⇒ 배열 연산을 빠르게 처리

e.g., Computing Reciprocals



- 파이썬 반복문과 NumPy 유니버설 함수 비교
 - 예: 역수 계산 in Python

```
def compute_reciprocals(values):  
    output = np.empty(len(values))  
  
    for i in range(len(values)):  
        output[i] = 1.0 / values[i]  
  
    return output
```

```
big_array = np.random.randint(1, 100, size=1000000)
```

```
%timeit compute_reciprocals(big_array)
```

코드 실행시간 측정

1.45 s $\pm$ 7.1 ms per loop (mean $\pm$ std. dev. of 7 runs, 1 loop each)

e.g., Computing Reciprocals



- 파이썬 반복문과 NumPy 유니버설 함수 비교
 - 예: 역수 계산 in NumPy

```
big_array = np.random.randint(1, 100, size=1000000)
```

```
%timeit 1 / big_array
```

2.03 ms $\pm$ 70.3 μ s per loop (mean $\pm$ std. dev. of 7 runs, 100 loops each)

1ms = 1,000 μ s

Why Does Vectorized Code Run Faster ?



not vectorized

a		b
1	*	6
2	*	7
3	*	8
4	*	9
5	*	10

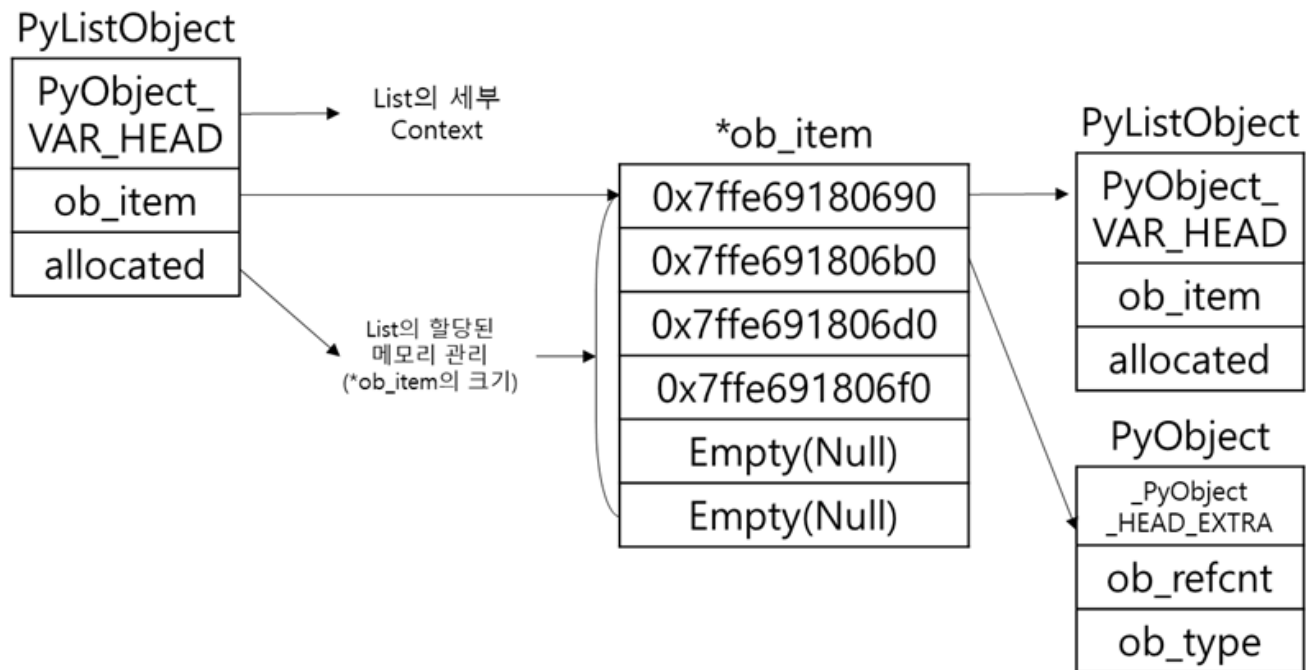
5 operations

vectorized

a		b
1		6
2		7
3	*	8
4		9
5	*	10

2 operations

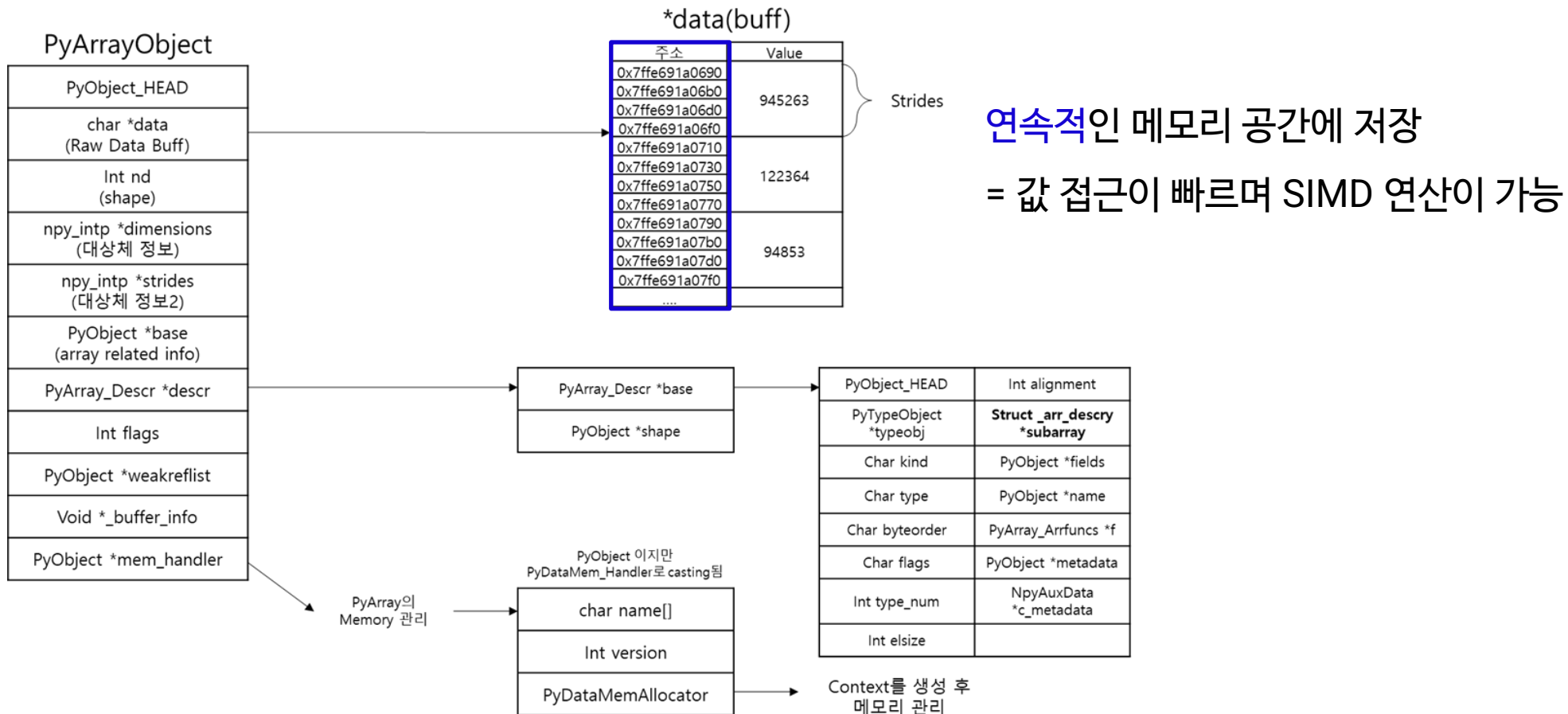
• Python Lists



불연속적인 메모리 공간에 저장
= 포인터 참조를 통해 값에 접근

NumPy Arrays

- 컴퓨터의 기계어 명령 machine instruction은 보통 하나의 데이터에 대한 연산을 수행
- SIMD Single Instruction Multiple Data: 하나의 기계어 명령으로 동일한 타입의 여러 데이터를 한 번에 처리 (병렬 처리 기법)



Universal Functions (UFuncs)



- 유연한 연산 제공: 스칼라-배열, 배열-배열, 다차원 배열 연산 등
 - 유니버설 함수의 벡터화 연산은 파이썬 루프로 구현된 연산보다 대부분 더 효율적

```
arr1 = arr1 - 2 # array + scalar operation
print(arr1, arr2)

arr3 = arr1 + arr2 # array + array operation
print(arr3)

arr4 = np.random.randint(10, size=(3, 5))
print(arr4)

arr5 = arr4 ** 2 # multi-dimensional array operation
print(arr5)
```

- 파이썬 반복문을 작성할 때 벡터화 연산으로 대체 가능한지 검토할 필요

UFuncs: Array Arithmetic

- 배열의 사칙연산: 파이썬 기본 산술 연산자 또는 **UFunc** 사용

```
x = np.arange(4)
```

생성

[0] [1] [2] [3]

0	1	2	3
---	---	---	---

`x + 3` 또는 `np.add(x, 3)`

+3 +3 +3 +3

[0] [1] [2] [3]

3	4	5	6
---	---	---	---

UFuncs: Array Arithmetic



표. 산술 연산자와 대응되는 유니버설 함수들

연산자	대응 ufuncs	설명
+	<code>np.add()</code>	덧셈(예: $1 + 1 = 2$)
-	<code>np.subtract()</code>	뺄셈(예: $3 - 2 = 1$)
-	<code>np.negative()</code>	단항 음수(예: -2)
*	<code>np.multiply()</code>	곱셈(예: $2 * 3 = 6$)
/	<code>np.divide()</code>	나눗셈(예: $3 / 2 = 1.5$)
//	<code>np.floor_divide()</code>	바닥 나눗셈(예: $3 // 2 = 1$)
**	<code>np.power()</code>	지수 연산(예: $2 ** 3 = 8$)
%	<code>np.mod()</code>	나머지 연산(예: $9 \% 4 = 1$)

UFuncs: Absolute Value



- `np.abs()`: 절댓값 함수
 - Python에서 `abs()`을 기본으로 제공하던데 ? $\Rightarrow$ 벡터화 연산!

```
x = np.array([-2, -1, 0, 1, 2])  
np.abs(x)
```

```
[2 1 0 1 2]
```

- 복소수의 절댓값 계산

```
x = np.array([3-4j, 4-3j, 2+0j, 0+1j])  
np.abs(x)
```

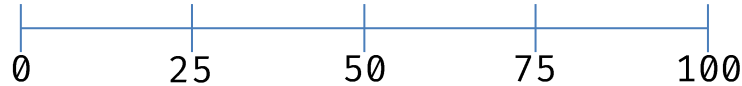
```
[5. 5. 2. 1.]
```

UFuncs: linspace()



- `np.linspace(start, stop, num)`: 선형(균일한) 간격의 벡터 생성
 - 다양한 벡터/행렬 생성에 자주 사용됨
 - `start`: 벡터의 시작 값
 - `stop`: 벡터의 종료 값
 - `num`: 생성할 값의 개수

```
x = np.linspace(0, 100, 5)
```



↓ 생성

[0] [1] [2] [3] [4]

0	25	50	75	100
---	----	----	----	-----

UFuncs: Trigonometric Functions



- 삼각함수: 데이터과학에서 자주 사용되는 함수
 - `np.sin()`: 사인 함수
 - `np.cos()`: 코사인 함수
 - `np.tan()`: 탄젠트 함수
 - 역삼각함수: `np.arcsin()`, `np.arccos()`, `np.arctan()`

UFuncs: etc.

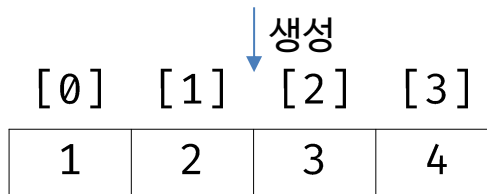


- 지수와 로그: `np.exp()`, `np.power()`, `np.log()`, `np.log10()` 등
- 그 외: 쌍곡선 삼각함수, 비트연산, 비교 연산자, 라디안 등
- `scipy.special`: 고급 수치연산을 유니버설 함수 형태로 제공하는 패키지
 - 감마함수, 가우스 적분, 보수/역수 등

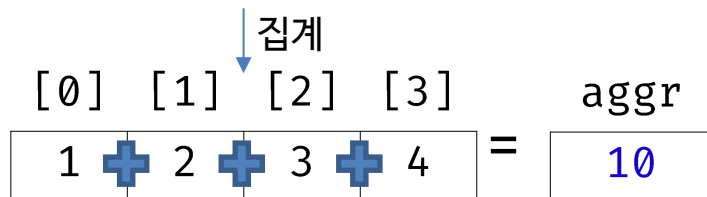
UFuncs: Aggregation

- 데이터의 기술통계^{descriptive statistics}를 위한 집계 함수 제공
- `reduce()`: 배열을 특정 연산으로 축소

```
x = np.arange(1, 5)
```



```
aggr = np.add.reduce(x)
```

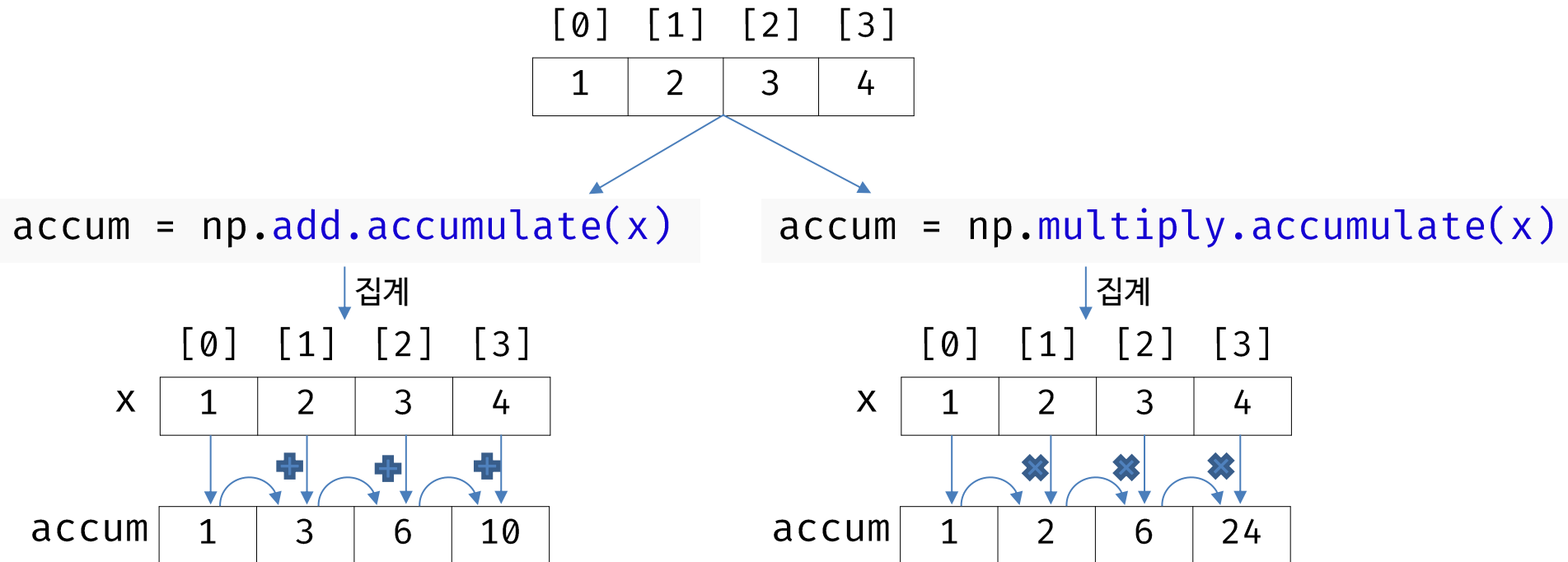


```
aggr = np.multiply.reduce(x)
```



UFuncs: Aggregation

- `accumulate()`: 계산의 **중간 결과들을 배열로 저장**



Summing the Values in an Array



- `np.sum()`: 총합 계산

```
arr = np.random.rand(1000000)  
np.sum(arr)
```

500422.19897015305

- 파이썬 내장함수 `sum()` 보다 연산이 훨씬 빠름

```
%timeit sum(arr)
```

111 ms $\pm$ 3.92 ms per loop (mean $\pm$ std. dev. of 7 runs, 10 loops each)

```
%timeit np.sum(arr)
```

345 μ s $\pm$ 22.7 μ s per loop (mean $\pm$ std. dev. of 7 runs, 1000 loops each)

1ms = 1000 μ s

Other Aggregation Functions



- 그 외의 집계함수

유니버설 함수	유니버설 함수 (누락 값 처리)	설명
<code>np.sum()</code>	<code>np.nansum()</code>	배열 원소 합
<code>np.prod()</code>	<code>np.nanprod()</code>	배열 원소 곱
<code>np.max()</code>	<code>np.nanmax()</code>	최댓값
<code>np.min()</code>	<code>np.nanmin()</code>	최솟값
<code>np.mean()</code>	<code>np.nanmean()</code>	평균값
<code>np.median()</code>	<code>np.nanmedian()</code>	중앙값
<code>np.percentile()</code>	<code>np.nanpercentile()</code>	하위 %의 원소
<code>np.var()</code>	<code>np.nanvar()</code>	분산
<code>np.std()</code>	<code>np.nanstd()</code>	표준 편차
<code>np.argmax()</code>	<code>np.nanargmax()</code>	최댓값 원소 인덱스
<code>np.argmin()</code>	<code>np.nanargmin()</code>	최솟값 원소 인덱스
<code>np.any()</code>	-	참을 만족하는 원소가 있는지
<code>np.all()</code>	-	모든 원소들이 참을 만족하는지

Multi Dimensional Aggregates

- 다차원 집계: 특정 차원을 기준으로 집계

– `axis` 속성

arr [0] [1] [2] [3]

[0]	3	6	6	0
[1]	9	8	4	7
[2]	0	0	7	1

`np.sum(arr, axis=0)`

	[0]	[1]	[2]	[3]
[0]	3	6	6	0
[1]	9	8	4	7
[2]	0	0	7	1
	12	14	17	8

Diagram illustrating the calculation of the sum along axis 0. A vertical blue arrow with a plus sign indicates the summation of elements across rows for each column.

`np.sum(arr, axis=1)`

	[0]	[1]	[2]	[3]
[0]	3	6	6	0
[1]	9	8	4	7
[2]	0	0	7	1
	=	=	=	
	15	28	8	

Diagram illustrating the calculation of the sum along axis 1. A horizontal blue arrow with a plus sign indicates the summation of elements across columns for each row.

3.2 NumPy 고급 기능

- 브로드캐스팅
- 마스킹
- 팬시 인덱싱

Flexible Operations by NumPy



- 유연한 연산 제공: 스칼라-배열, 배열-배열, 다차원 배열 연산 등

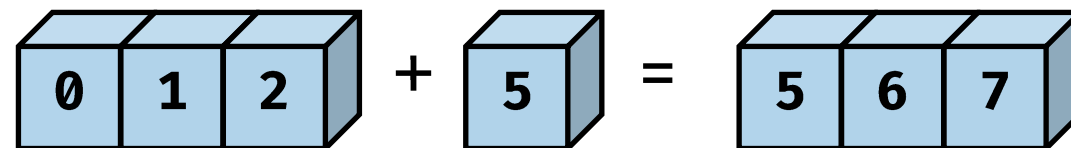
```
arr1 = arr1 - 2 # array + scalar operation  
print(arr1, arr2)
```

```
arr3 = arr1 + arr2 # array + array operation  
print(arr3)
```

```
arr4 = np.random.randint(10, size=(3, 5))  
print(arr4)
```

```
arr5 = arr4 ** 2 # multi-dimensional array operation  
print(arr5)
```

어떻게?





Casting

(주물)

쇳물을 거푸집에 부어
원하는 모양으로 만들기

Broadcasting

- **브로드캐스팅: 다른 크기의 배열간 이항 연산**을 수행하는 기능
 - 유니버설 함수에서 지원
 - 예: 배열-스칼라 연산



`a = np.array([0, 1, 2])`
`a + 5`

=

`a = np.array([0, 1, 2])`
`b = np.array([5, 5, 5])`
`a + b`

- 브로드캐스팅의 장점: **값 복제가 발생하지 않음**

👻 브로드캐스팅의 원리는 개념적으로 "확장"하는 것처럼 보이지만, 실제로 메모리에서 값이 복제되는 것은 아닙니다. 내부적으로는 동일한 스칼라(또는 배열) 값을 여러 번 재사용하는 방식으로 동작합니다. 이 방식은 메모리 효율성을 높이며, 연산의 속도를 향상시킵니다.

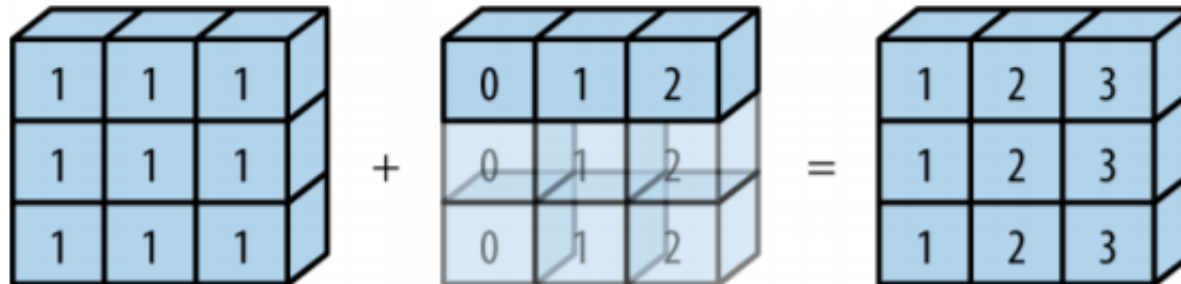
ex) 1D Array + 2D Array

- 예제: 1차원 배열 + 2차원 배열

- 1차원 배열 $\Rightarrow$ 2차원 배열로 브로드캐스팅 후 연산 수행

```
a = np.ones((3, 3))  
b = np.arange(3)
```

a + b

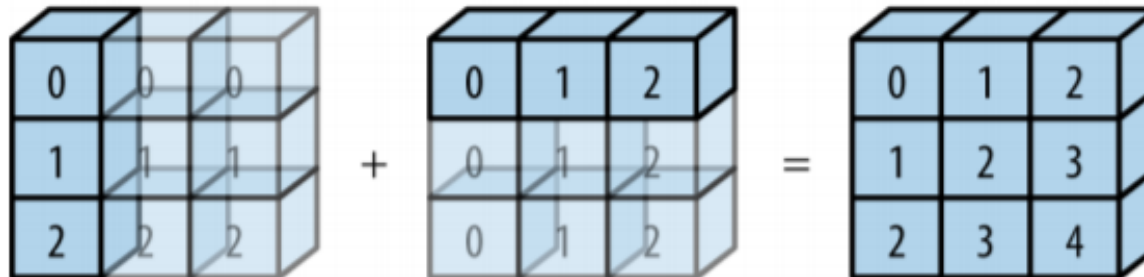


ex) Broadcast Both Arrays

- 예제: 두 배열 모두 브로드캐스팅
 - 두 배열 모두 공통 형상^{common shape}으로 브로드캐스팅

```
a = np.arange(3)  
a = a[:, np.newaxis]  
b = np.arange(3)
```

a + b



Broadcasting Rules



- 브로드캐스팅은 다음의 규칙들을 따름
 - 규칙 1: 두 배열의 차원수가 다른 경우
 - ⇒ 더 작은 차원의 배열에 차원을 추가 (형상 왼쪽에 1 추가)
 - 규칙 2: 두 배열의 형상이 특정 차원에서 일치하지 않은 경우
 - ⇒ 해당 차원이 1인 배열을 다른 배열의 형상과 일치하도록 증가
 - 규칙 3: 임의의 차원에서 크기가 일치하지 않고 1도 아닌 경우
 - ⇒ 브로드캐스팅 불가

ex) 1D Array + 2D Array

```
a = np.ones((2, 3))  
b = np.arange(3)
```

– 두 배열의 형상

```
a.shape = (2, 3)  
b.shape = (3,)
```

– 규칙 1에 따라, 배열 **b**에 차원 추가

```
a.shape → (2, 3)  
b.shape → (1, 3)
```

– 규칙 2에 따라, 배열 **b**의 첫 번째 차원을 배열 **a**와 일치하도록 증가

```
a.shape → (2, 3)  
b.shape → (2, 3)
```

a + b

```
[[1. 2. 3.]  
 [1. 2. 3.]
```

ex) Broadcast both Arrays



```
a = np.arange(3)
a = a.reshape((3, 1))
b = np.arange(3)
```

– 두 배열의 형상

```
a.shape = (3, 1)
b.shape = (3,)
```

– 규칙 1 적용

```
a.shape → (3, 1)
b.shape → (1, 3)
```

– 규칙 2 적용

```
a.shape → (3, 3)
b.shape → (3, 3)
```

a + b

```
[[0 1 2]
 [1 2 3]
 [2 3 4]]
```

ex) Incompatible Case



```
a = np.ones((3, 2))  
b = np.arange(3)
```

– 두 배열의 형상

```
a.shape = (3, 2)  
b.shape = (3,)
```

– 규칙 1 적용

```
a.shape → (3, 2)  
b.shape → (1, 3)
```

– 규칙 2 적용

- a의 두 번째 차원의 경우 **규칙 2 적용 불가능**

```
a.shape → (3, 2)  
b.shape → (3, 3)
```

ValueError: operands could not be broadcast together with shapes (3,2) (3,)

Recap: Comparison Operators

- 배열의 비교 연산(마스킹) $\Rightarrow$ Boolean 배열 Boolean array 출력

```
x = np.array([1, 2, 3, 4, 5])  
x < 3
```

```
[True, True, False, False, False]
```

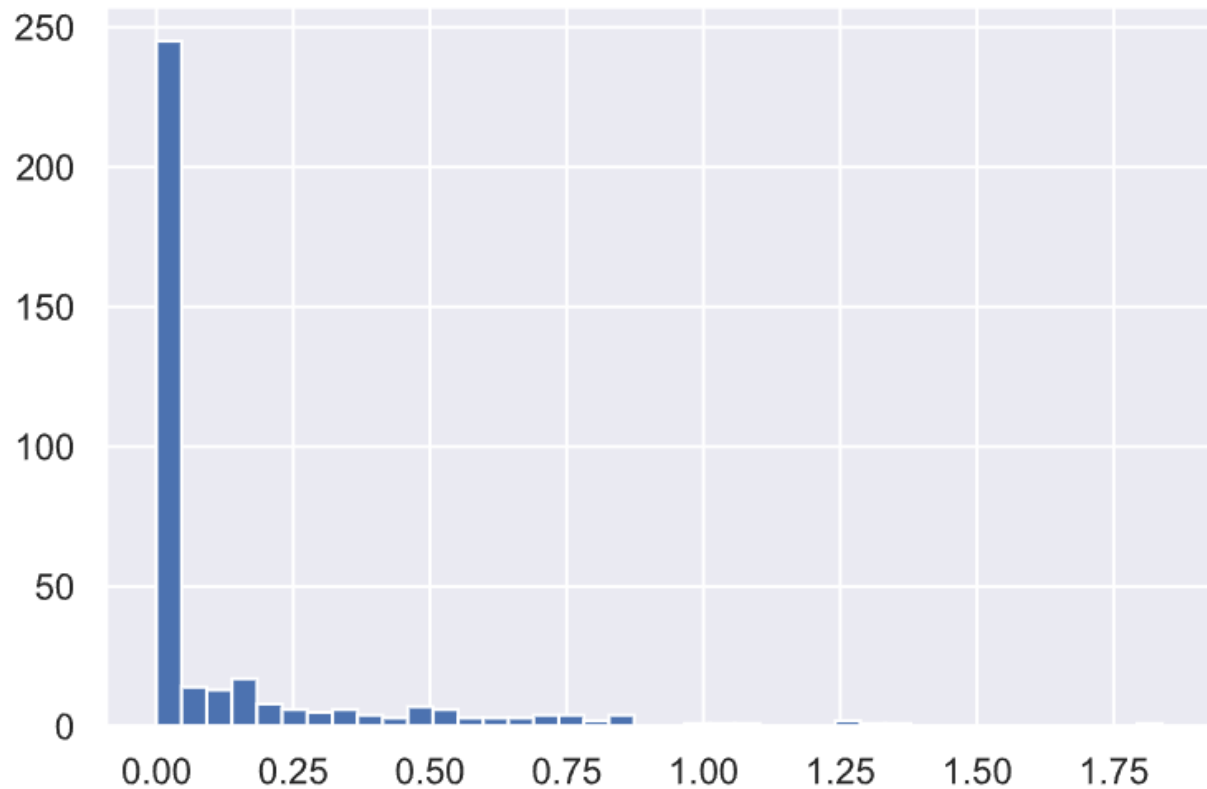
```
x = np.array([1, 2, 3, 4, 5])  
x  $\geq$  3
```

```
[False False True True True]
```

- Boolean 배열(또는 마스크): 배열의 값을 추출, 수정, 계산, 조작할 때 사용

Example: Counting Rainy Days

- 예제: 시애틀의 2014년 강수량 히스토그램



히스토그램을 통해 알 수 있는 것:

- 대부분 날에 비가 내리지 않음

히스토그램만으로는 알 수 없는 것:

- 비가 온 날은 며칠일까?
- 비가 온 날의 평균 강수량은?
- 0.5인치 이상 비가 온 날은 며칠?

⇒ Boolean 배열을 이용한 연산이 필요

UFuncs for Comparison Operations



- 비교 연산을 위한 유니버설 함수

연산자	대응 ufunc
$=$	<code>np.equal()</code>
$\neq$	<code>np.not_equal()</code>
$<$	<code>np.less()</code>
$\leq$	<code>np.less_equal()</code>
$>$	<code>np.greater()</code>
$\geq$	<code>np.greater_equal()</code>

UFuncs for Comparison Operations



- 다차원 배열의 비교 연산

```
np.random.seed(0)
x = np.random.randint(10, size=(3, 4))
print(x)
```

```
[[5 0 3 3]
 [7 9 3 5]
 [2 4 7 6]]
```

```
x < 6
```

```
[[ True  True  True  True]
 [False False  True  True]
 [ True  True False False]]
```

Boolean Logic



- Boolean 배열을 이용한 연산 예: 원소 개수 세기

- Boolean 의 정수 표현: True: 1, False: 0

- `np.count_nonzero(x < 6)`

- 8

- `np.sum(x < 6)`

- 8

- `np.any()` 예: 8보다 큰 값이 하나라도 있는가?

- `np.any(x > 8)`

- True

- `np.all()` 예: 모든 값이 1보다 큰가?

- `np.all(x > 1)`

- False

Data Selection Through Masking Operations



- 마스킹 연산: Boolean 배열을 마스크로 사용하여 True를 만족하는 값들을 선택

	x			
	[0]	[1]	[2]	[3]
[0]	5	0	3	3
[1]	7	9	3	5
[2]	2	4	7	6

$x < 5$

```
[[False True True True]
 [False False True False]
 [ True True False False]]
```

$x[x < 5]$

```
[0 3 3 3 2 4]
```

Fancy Indexing



- 지금까지 배운 배열 원소 접근 방법:
 - 인덱싱(예: $x[0]$), 슬라이싱(예: $x[2:7]$), 마스크 연산(예: $x[x > 0]$)
- **팬시 인덱싱**: 한 번에 여러 배열 원소에 접근하는 방법
 - 단일 값^{scalar} 대신 인덱스 배열을 전달

Fancy Indexing

```
x = np.random.randint(100, size=10)
```

↓ 배열 생성

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]
37	12	72	9	75	5	79	64	16	1

70대 정수 값 선택하기: ① 기본 문법

② 팬시 인덱싱

```
a = x[2]; b = x[4]; c = x[6]
print(a, b, c)
```

```
ind = [2, 4, 6]
print(x[ind])
```

[0]	[1]	[2]
72	75	79

- 다차원 배열에 대한 팬시 인덱싱

	[0]	[1]	[2]	[3]	[4]
[0]	8	8	6	2	8
[1]	7	2	1	5	4
[2]	4	5	7	3	6



```
row = [0, 1, 2]  
col = [2, 1, 4]  
x[row, col]
```

두 개의 인덱스(행, 열)를 이용한
팬시 인덱싱



	[0]	[1]	[2]	[3]	[4]	
[0]	8	8	6	2	8	→
[1]	7	2	1	5	4	
[2]	4	5	7	3	6	

[0]	[1]	[2]
6	2	6

- 다차원 배열에 대한 팬시 인덱싱 + 브로드캐스팅 = 다차원 배열

	[0]	[1]	[2]	[3]	[4]
[0]	8	8	6	2	8
[1]	7	2	1	5	4
[2]	4	5	7	3	6



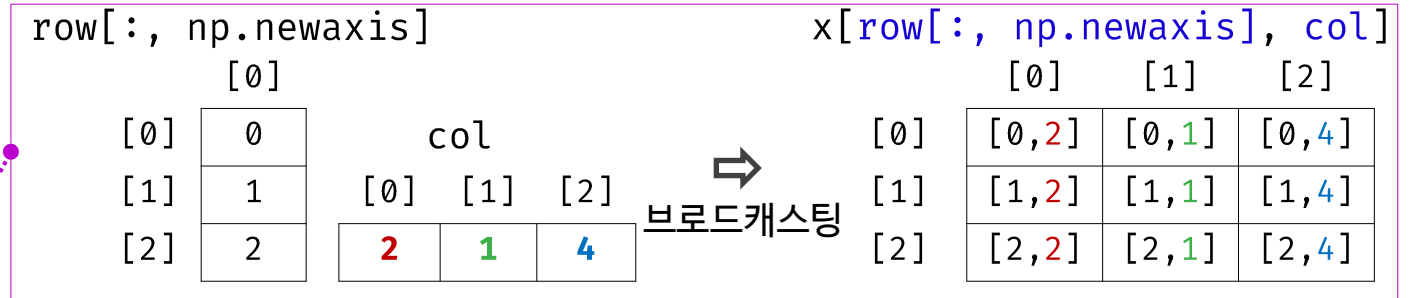
```
row = np.array([0, 1, 2])
col = np.array([2, 1, 4])
x[row[:, np.newaxis], col]
```



	[0]	[1]	[2]	[3]	[4]
[0]	8	8	6	2	8
[1]	7	2	1	5	4
[2]	4	5	7	3	6



	[0]	[1]	[2]
[0]	6	8	8
[1]	1	2	4
[2]	7	5	6



Combinations of Indexing Methods

- 다른 인덱싱 방법과 결합된 팬시 인덱싱

- 기본 인덱싱 + 팬시 인덱싱

```
x[2, [2, 0, 1]]  
  
[11  9 10]
```

	[0]	[1]	[2]	[3]
[0]	1	2	3	4
[1]	5	6	7	8
[2]	9	10	11	12

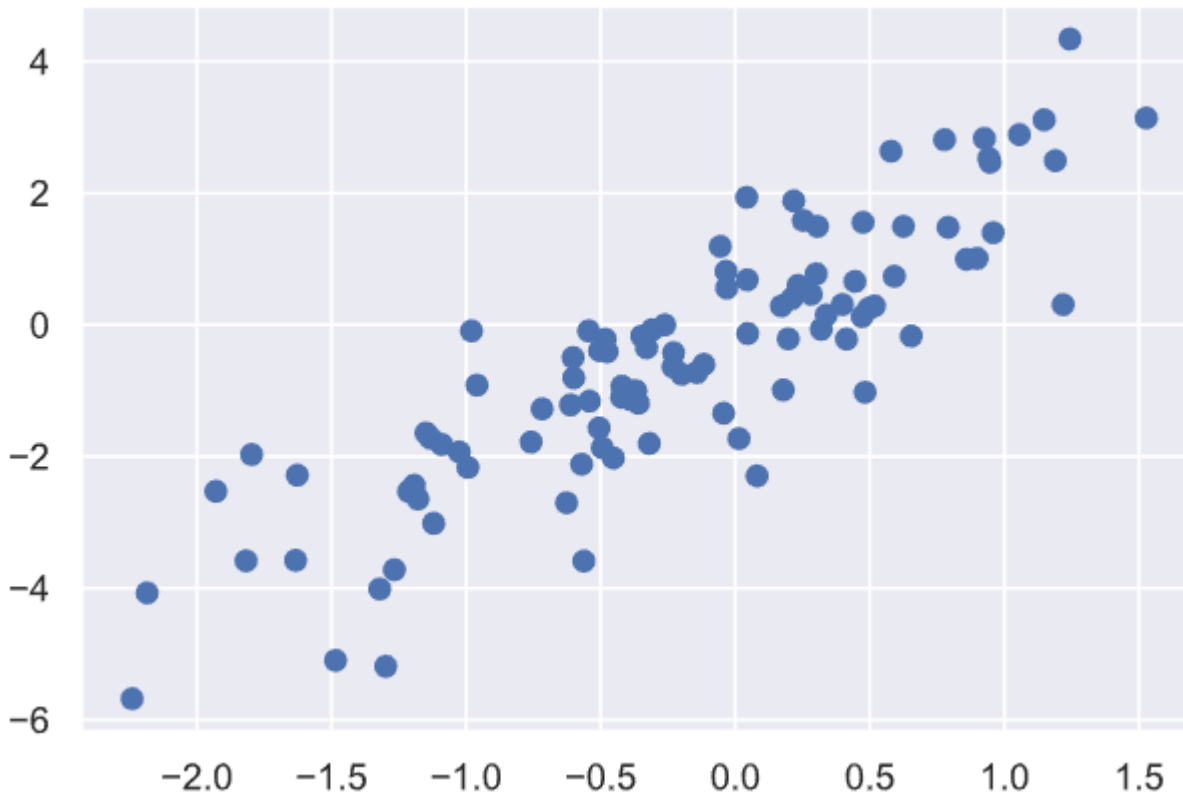
- 슬라이싱 + 팬시 인덱싱

```
x[1:, [2, 0, 1]]  
  
[[ 7  5  6]  
 [11  9 10]]
```

	[0]	[1]	[2]	[3]
[0]	1	2	3	4
[1]	5	6	7	8
[2]	9	10	11	12

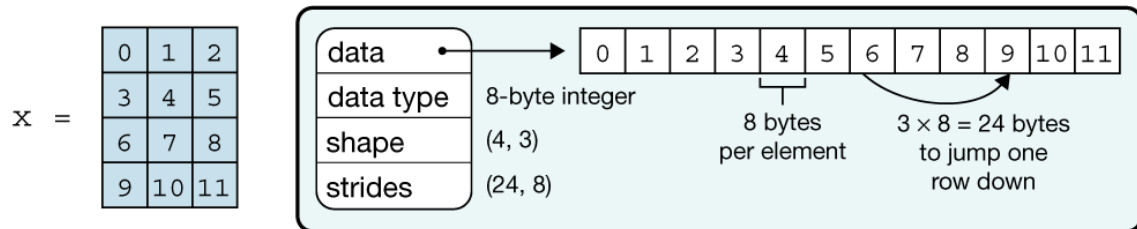
ex) Selecting Random Points

- 예제: 팬시 인덱싱을 통한 임의의 점 집합 선택하기

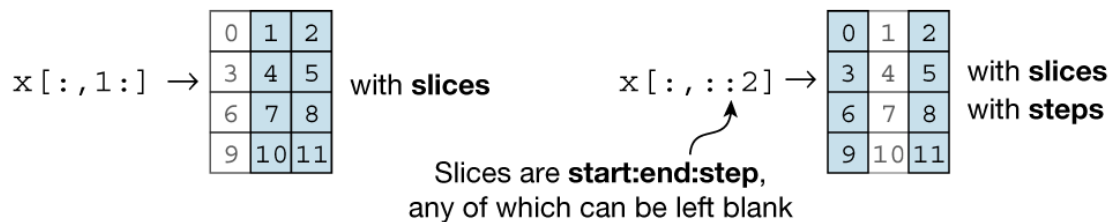


Summary of NumPy

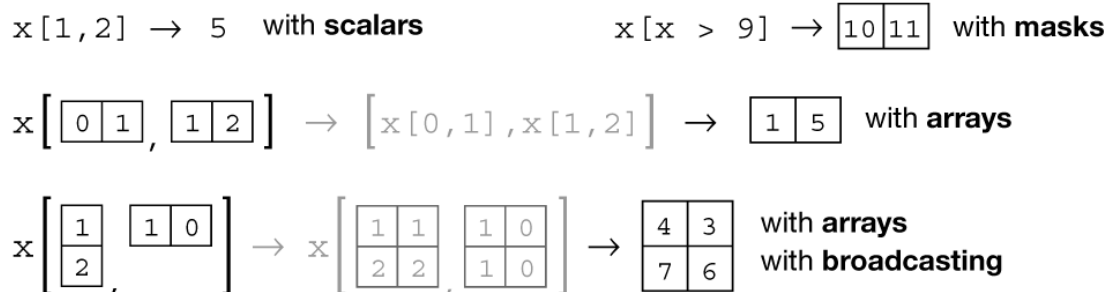
a Data structure



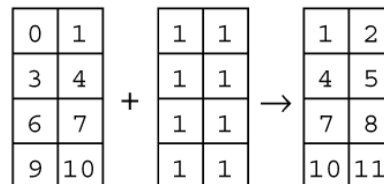
b Indexing (view)



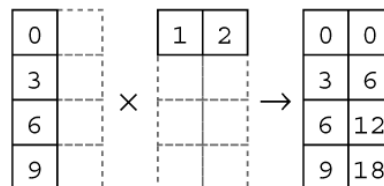
c Indexing (copy)



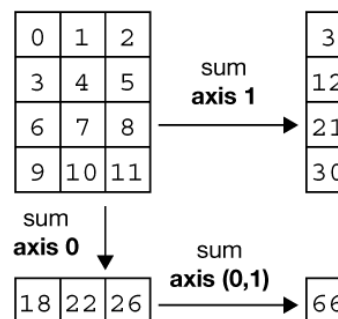
d Vectorization



e Broadcasting



f Reduction



g Example

```
In [1]: import numpy as np
```

```
In [2]: x = np.arange(12)
```

```
In [3]: x = x.reshape(4, 3)
```

```
In [4]: x
```

```
Out [4]:
```

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

```
In [5]: np.mean(x, axis=0)
```

```
Out [5]: array([4.5, 5.5, 6.5])
```

```
In [6]: x = x - np.mean(x, axis=0)
```

```
In [7]: x
```

```
Out [7]:
```

```
array([[ -4.5,  -4.5,  -4.5],
       [ -1.5,  -1.5,  -1.5],
       [  1.5,   1.5,   1.5],
       [  4.5,   4.5,   4.5]])
```

Conclusions



- NumPy: 수치연산에 최적화된 라이브러리
 - 벡터화된 연산: 코드 실행 속도 증가 및 메모리 절약
- 브로드캐스팅: 다른 크기의 배열간 연산 가능
 - 유니버설 함수를 더 쉽게 사용
- 마스킹 연산: Boolean 배열을 이용한 배열 원소 접근
- 팬시인덱싱: 인덱스 배열을 이용한 배열 원소 접근

References



- Harris, C. R., et al., “Array Programming with NumPy,” Nature, Vol. 585, No. 7825, pp. 357-362, 2020 ([link](#))
- PyCon 2015, Losing your Loops: Fast Numerical Computing with NumPy, YouTube ([link](#))
- NumPy Illustrated: The Visual Guide to NumPy, MYBLUELINUX.COM ([link](#))
- Teus, Numpy가 빠른 이유-2편 (PyObject와 메모리 구조 관점에서), DEVOCEAN ([link](#))
- VanderPlas, J., Python Data Science Handbook, GitHub ([link](#))
- freeCodeCamp.org, Python NumPy Tutorial for Beginners, YouTube ([link](#))
- Konrad, M., VECTORIZATION AND PARALLELIZATION IN PYTHON WITH NUMPY AND PANDAS, WZB Data Science Blog ([link](#))
- Why does vectorized code run faster than looped code?, reddit ([link](#))
- VanderPlas, J., Why Python is Slow: Looking Under the Hood, GitHub Page ([link](#))
- 김정인, 파이썬 데이터 사이언스 핸드북, 위키북스, 2017.



수고하셨습니다.